Project Iteration 2: Detailed User Stories, Requirements, and Initial Prototype

Course: DIGT2107, Fall Term 2026
Instructor: Dr. May Haidar
Due Date: October 11, 2026

---

1. Introduction

Project Name: SplitEven
GitHub Repository: https://github.com/Edddro/DIGT2107-SplitEven
Team Number: Team 2
Team Members:

- Adam Bridges
- Edward Drobnis
- Francis Kafieh
- Claire Bhagat

Document Overview: This document outlines the deliverables for Iteration 2 of the project, focusing on defining detailed user stories, specifying functional and non-functional requirements, and developing an initial prototype. This iteration also emphasizes proper documentation and planning, which sets the stage for subsequent development phases. The goal is to establish a solid foundation by clarifying what the system should do and beginning the implementation with a functional prototype.

---

2. Functional and Non-Functional Requirements

This section specifies the key behaviours and performance expectations of the software.

2.1. Functional Requirements

Each requirement lists the user story it comes from.

Group and Expense Management (GE)

- **Requirement 1 (FR-001):** The system shall allow a user to create a group with a name. *(GE-001)*
- **Requirement 2 (FR-002):** The system shall allow a user to add a member to a group by name and shall assign each member a unique identifier so that members with the same name can be distinguished. The system shall reject a blank member name. *(GE-001)*
- **Requirement 3 (FR-003):** The system shall allow a member to be removed from a group only when their current balance is zero. If the member has an outstanding balance, the system shall prevent their removal and display the amount still owed or owed to them. *(GE-005)*
- **Requirement 4 (FR-004):** The system shall allow a user to record an expense with an amount, a payment method, a date, a category, an optional note, and the members who share it. *(GE-002)*
- **Requirement 5 (FR-005):** The system shall split an expense equally among the members who share it. *(GE-003)*
- **Requirement 6 (FR-006):** The system shall split an expense by custom amounts and shall not save the expense unless the amounts add up exactly to the total. *(GE-003)*
- **Requirement 7 (FR-007):** The system shall split an expense by percentage, shall not save the expense unless the percentages add up to 100, and shall round shares so they always add up exactly to the total. *(GE-003)*
- **Requirement 8 (FR-008):** The system shall check that an expense has an amount greater than zero, a payment method, a valid date, and at least one member sharing it before saving. *(GE-002, UI-002)*
- **Requirement 9 (FR-009):** The system shall allow a user to edit or delete an expense and shall recalculate all affected member balances after the change. *(GE-004)*
- **Requirement 10 (FR-010):** The system shall save all groups, members, expenses, and payments to files on the user's computer after every change, and shall load them when the application starts. *(GE-006)*

Balances and Settling Up (BS)

- **Requirement 11 (FR-011):** The system shall calculate each member's balance (what they owe or are owed) from all expenses and payments in the group. *(BS-001)*

- **Requirement 12 (FR-012):** The system shall show which members owe money to other members and the amount owed based on the current balances. *(BS-002)*
- **Requirement 13 (FR-013):** : The system shall allow a user to record a payment between two members of the same group and shall not allow a payment greater than the amount the paying member currently owes the receiving member. *(BS-003)*
- **Requirement 14 (FR-014):** The system shall allow a user to edit or delete a payment, and shall recalculate all balances afterwards. *(BS-004)*

User Experience and Interface (UI)

- **Requirement 15 (FR-015):** The system shall show an error message next to the field that caused it when input is missing or invalid, and a confirmation message when an action succeeds. *(UI-002)*
- **Requirement 16 (FR-016):** The system shall show the group's current balances on the main screen when the application opens. *(UI-001)*

Reporting and Summaries (RS)

- **Requirement 17 (FR-017):** The system shall show a table of past expenses and allow it to be filtered by member, category, and date range. *(RS-001)*
- **Requirement 18 (FR-018):** The system shall show total spending per category and per member for a chosen period. *(RS-002)*
- **Requirement 19 (FR-019):** The system shall save a plain text summary of the current balances and who owes whom to a file chosen by the user. *(RS-003)*

2.2. Non-Functional Requirements

- **Performance (NFR-001):** Balances update on screen within 1 second of saving an expense or payment, for a group of up to 20 members and 1,000 expenses.
- **Usability (NFR-002):** A first-time user can add an expense in under one minute without referring to instructions.
- **Accuracy (NFR-003):** Money is stored in whole cents rather than decimals. Split shares always add up exactly to the expense total, and all balances in a group always add up to zero.
- **Reliability (NFR-004):** Every confirmed change is saved to disk straight away, so closing the application never loses saved data.
- **Privacy (NFR-005):** The application makes no network connections. All data stays in local files, with no accounts and no bank connections.
- **Accessibility (NFR-006):** Every main action can be completed using the keyboard alone.
- **Maintainability (NFR-007):** Calculation and validation code does not depend on the user interface, so it can be unit tested on its own.

- **Scalability (NFR-008):** The application shall access persistent data through a single storage interface so that the CSV-based storage can later be replaced by another storage system without requiring changes to the calculation logic.
- **Portability (NFR-009):** The application runs on Windows, macOS, and Linux with a current Java LTS version installed.

Requirements Traceability

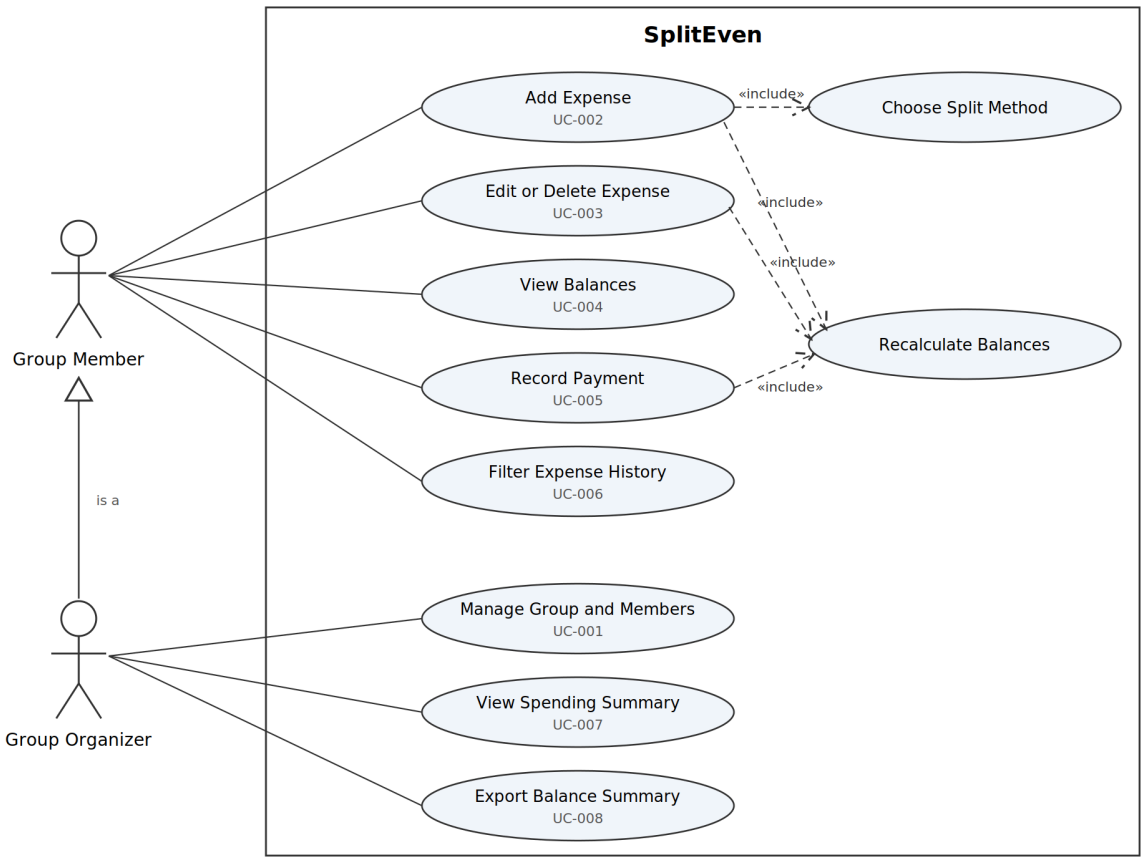
Requirement	**User story**	**Use case**
FR-001, FR-002	GE-001	UC-001
FR-003	GE-005	UC-001
FR-004, FR-008	GE-002	UC-002
FR-005, FR-006, FR-007	GE-003	UC-002
FR-009	GE-004	UC-003
FR-010	GE-006	All
FR-011, FR-012	BS-001, BS-002	UC-004
FR-013	BS-003	UC-005
FR-014	BS-004	UC-005
FR-015	UI-002	UC-002, UC-005
FR-016	UI-001	UC-004
FR-017	RS-001	UC-006
FR-018	RS-002	UC-007
FR-019	RS-003	UC-008

---

3. Use Cases

Use cases provide a structured approach to understanding the interactions between users and the system.

3.1. Use Case Diagram



There are two actors. A **Group Member** records expenses and payments and checks balances. A **Group Organizer** is a member who also sets up the group and produces summaries, so the organizer can do everything a member can.

Adding an expense, editing or deleting one, and recording a payment all include **Recalculate Balances**, because every one of them changes what people owe. Adding an expense also includes the **Choose Split Method**.

3.2. Use Case Descriptions

1. Use Case Name: Manage Group and Members (UC-001)

Actors: Group Organizer
Description: This use case describes how the organizer creates a group, adds members, and removes members.
Preconditions: The application is open.
Main Flow:

- The organizer selects "New Group" and enters a group name.
- The system checks that the name is not blank and creates the group.
- The organizer selects "Add Member" and enters a member's name.
- The system checks that the name is not blank and assigns the member a unique identifier.
- To remove a member, the organizer selects the member and chooses "Remove".
- The system checks that the member's balance is zero. If it is, the member is removed. If not, the system blocks the removal and shows how much is still owed.

Postconditions: The group and its member list are saved and shown on the main screen.

2. Use Case Name: Add Expense (UC-002)

Actors: Group Member, Group Organizer
Description: This use case describes how a member records an expense and how it is split.
Preconditions: A group exists with at least two members.
Main Flow:

- The member selects "Add Expense".
- The system shows the expense form.
- The member enters the amount, payment method, date, category, and an optional note, and selects the members who share the expense.
- The member chooses a split method: equal, custom amounts, or percentages, and enters the amounts or percentages if needed.
- The member selects "Save".
- The system validates the input. If anything is missing or invalid (for example, custom amounts that do not add up to the total), the system displays an error message next to the relevant field and does not save the expense.
- If the input is valid, the system calculates each person's share, rounding so the shares add up exactly to the total, saves the expense, and recalculates balances.
- The system shows a confirmation message.

Postconditions: The expense is saved and the updated balances are shown on the main screen.

3. Use Case Name: Edit or Delete Expense (UC-003)

Actors: Group Member, Group Organizer
Description: This use case describes how a member corrects or removes an expense entered by mistake.
Preconditions: At least one expense exists in the group.
Main Flow:

- The member opens the expense history and selects an expense.
- The member chooses "Edit" or "Delete".
- For an edit, the system shows the form filled in with the current details. The member makes changes and saves, and the system checks the input in the same way as when adding an expense.
- For a delete, the system asks the member to confirm.
- The system saves the change and recalculates balances.

Postconditions: The expense is updated or removed, and the balances reflect the change.

4. Use Case Name: View Balances (UC-004)

Actors: Group Member, Group Organizer
Description: This use case describes how a member checks what they owe or are owed, and who owes whom.
Preconditions: A group exists.
Main Flow:

- The member opens the application.
- The system loads the saved data and shows the main screen.
- The member selects a group to open the group screen.
- The group screen lists each member's balance, showing whether they owe money or are owed money.
- The group screen also lists who owes whom and how much.
- If a group has no expenses yet, the system shows a message saying everyone is settled.

Postconditions: No data changes. The member can see the group's current position.

5. Use Case Name: Record Payment (UC-005)

Actors: Group Member, Group Organizer
Description: This use case describes how a member records money paid back to another member, including correcting a payment entered by mistake.
Preconditions: At least one member owes money to another.
Main Flow:

- The member selects "Record Payment".
- The system shows who owes whom.
- The member selects who paid, who received the payment, and the amount.

- The system checks that the amount is greater than zero and not more than what is currently owed. If the amount is too large, it shows the maximum allowed and saves nothing.
- The system saves the payment, recalculates balances, and shows a confirmation.
- To correct a payment, the member selects it from the payment list and edits or deletes it, and the system recalculates balances.

Postconditions: The payment is saved, and balances show the reduced amount owed.

6. Use Case Name: Filter Expense History (UC-006)

Actors: Group Member, Group Organizer
Description: This use case describes how a member looks at part of a group's spending.
Preconditions: At least one expense exists in a group.
Main Flow:

- The member selects the expense history screen view.
- A side panel opens and reveals a table of all expenses.
- The member can filter by a group, a member, a category, a date range, or any combination.
- The system shows only the matching rows of expenses and their total.
- The member clears the filters to see every expense again.

Postconditions: No data changes.

7. Use Case Name: View Spending Summary (UC-007)

Actors: Group Organizer
Description: This use case describes how the organizer sees where the group's money went over a period.
Preconditions: At least one expense exists in a group.
Main Flow:

- The organizer selects "Summary".
- The organizer chooses a start and end date.
- The system shows total spending per category and per member for that period.

Postconditions: No data changes.

8. Use Case Name: Export Balance Summary (UC-008)

Actors: Group Organizer
Description: This use case describes how the organizer saves the current balances to share with members who are not at the computer.
Preconditions: A group exists.
Main Flow:

- The organizer selects "Export Balances".
- The system asks where to save the file.

- The system writes a plain text file listing today's date, each member's balance, and who owes whom.
- The system confirms where the file was saved.

Postconditions: A text file with the current summary exists on the user's computer.

---

4. Detailed User Stories

This section presents the refined user stories, breaking down high level features into specific, actionable stories.

4.1. Category 1: Group and Expense Management (GE)

1. User Story GE-001: Create a Group and Add Members

- **As an** organizer, **I want to** create a group and add members **so that** we can start tracking shared costs.
- **Acceptance Criteria:**
  - A group can be created with a name, and a blank name is rejected with a message.
  - Members can be added by name, and a blank name is rejected with a message.
  - The new group and its members appear on the main screen.
- **Priority:** High
- **Story Points:** 3

2. User Story GE-002: Add an Expense

- **As a** member, **I want to** add an expense with the amount, payment method, date, and category **so that** the group has a record of what was spent.
- **Acceptance Criteria:**
  - The form asks for amount, payment method, date, category, an optional note, and who shares the expense.
  - An amount of zero or less, a missing payment method, or no members selected is rejected with a message next to the field.
  - A valid expense is saved and appears in the expense history.
- **Priority:** High
- **Story Points:** 5

3. User Story GE-003: Choose How an Expense Is Split

- **As a** member, **I want to** split an expense equally, by amount, or by percentage **so that** uneven shares are recorded fairly.
- **Acceptance Criteria:**

    - An equal split divides the total evenly, and any leftover cent is assigned so the shares still add up to the total.
    - A custom split cannot be saved unless the amounts add up exactly to the total, and the message shows the difference.
    - A percentage split cannot be saved unless the percentages add up to 100.
    - For every split method, the saved shares add up exactly to the expense total.
- **Priority:** High
- **Story Points:** 3

4. User Story GE-004: Edit or Delete an Expense

- **As a** member, **I want to** edit or delete an expense I entered by mistake **so that** the records stay correct.
- **Acceptance Criteria:**
    - An edited expense is checked with the same rules as a new one.
    - Deleting an expense asks for confirmation first.
    - Balances update after every edit or delete.
- **Priority:** Medium
- **Story Points:** 5

5. User Story GE-005: Remove a Member

- **As an** organizer, **I want to** remove a member who has left the group **so that** the member list stays current.
- **Acceptance Criteria:**
    - A member with a balance of zero can be removed.
    - A member who owes or is owed money cannot be removed, and the message shows the amount still outstanding.
    - Past expenses involving a removed member stay in the history.
- **Priority:** Medium
- **Story Points:** 3

6. User Story GE-006: Keep Data After Closing

- **As a** member, **I want** the group's data saved automatically **so that** nothing is lost when I close the application.
- **Acceptance Criteria:**
    - Every saved group, member, expense, and payment is written to files straight away.
    - When the application is reopened, all data is loaded exactly as it was.
    - If no saved data exists, the application starts with an empty group list and no error.
- **Priority:** High
- **Story Points:** 5

4.2. Category 2: Balances and Settling Up (BS)

1. User Story BS-001: See My Balance

- **As a** member, **I want to** see my current balance **so that** I know whether I owe money or am owed money.
- **Acceptance Criteria:**
    - Each member's balance is shown as an amount owed or an amount owed to them.
    - The balance includes every expense and payment in the group.
    - All balances in the group add up to zero.
- **Priority:** High
- **Story Points:** 3

2. User Story BS-002: See Who Owes Whom

- **As a** member, **I want to** see who owes whom in the group **so that** we can agree on how to settle up.
- **Acceptance Criteria:**
    - The list shows each debt as "A owes B $X".
    - The debts listed exactly clear every member's balance when paid.
    - Members with a zero balance do not appear in the list.
- **Priority:** High
- **Story Points:** 3

3. User Story BS-003: Record a Payment

- **As a** member, **I want to** record a payment between two members **so that** the balances match money that has already been paid back.
- **Acceptance Criteria:**
    - A payment needs a payment method, a receiver, and an amount greater than zero.
    - A payment larger than what the payer owes the receiver is rejected, and the message shows the maximum allowed.
    - After saving, both members' balances update and a confirmation appears.
- **Priority:** High
- **Story Points:** 2

4. User Story BS-004: Correct a Payment

- **As a** member, **I want to** edit or delete a payment entered by mistake **so that** balances stay accurate.
- **Acceptance Criteria:**

    - An edited payment is checked with the same rules as a new one.
    - Deleting a payment asks for confirmation first.
    - Balances update after every edit or delete.
- **Priority:** Medium
- **Story Points:** 2

4.3. Category 3: User Experience and Interface (UI)

1. User Story UI-001: Balances on Opening

- **As a** member, **I want to** see the group's balances as soon as I open the app **so that** I can check where I stand quickly.
- **Acceptance Criteria:**
    - The main screen shows every member's balance and who owes whom when the application opens.
    - If there are no expenses yet, the screen says everyone is settled.
    - The main screen links to Add Expense, Settle Up, and Expense History.
- **Priority:** Medium
- **Story Points:** 8

2. User Story UI-002: Clear Error Messages

- **As a** member, **I want** clear messages when something I enter is missing or wrong **so that** I can fix it without guessing.
- **Acceptance Criteria:**
    - Each error message appears next to the field that caused it and says what to change.
    - Nothing is saved while any error is showing.
    - A confirmation message appears after every successful save.
    - Every form can be completed using only the keyboard.
- **Priority:** High
- **Story Points:** 5

4.4. Category 4: Reporting and Summaries (RS)

1. User Story RS-001: Filter Past Expenses

- **As a** member, **I want to** filter past expenses by member, category, and date range **so that** I can look at one part of our spending.
- **Acceptance Criteria:**
    - Filters can be used alone or together.
    - The table shows only matching expenses, with their total.
    - Clearing the filters shows every expense again.
- **Priority:** Medium
- **Story Points:** 5

2. User Story RS-002: Spending Totals by Category

- **As an** organizer, **I want** totals by category and by member for a chosen period **so that** the group can see where its money went.
- **Acceptance Criteria:**
    - The organizer can choose a start and end date.
    - Totals are shown per category and per member for that period.
    - The category totals add up to the total spending for the period.
- **Priority:** Low
- **Story Points:** 5

3. User Story RS-003: Save a Balance Summary

- **As an** organizer, **I want to** save a plain text summary of the balances **so that** I can share it with members who are not at the computer.
- **Acceptance Criteria:**
    - The file includes the date, each member's balance, and who owes whom.
    - The organizer chooses where the file is saved.
    - A confirmation shows where the file was saved.
- **Priority:** Low
- **Story Points:** 3

---

5. Initial Prototype Guidelines

5.1. Prototype Features

The prototype is the smallest version of SplitEven that works from start to finish: create a group, add an expense, and see the resulting balances. It covers the four highest priority stories that everything else depends on.

Prototype feature	**Stories**	**What it shows**
Create a group and add members	GE-001	Groups and members can be created and validated
Add an expense, split equally	GE-002, part of GE-003	Expenses are recorded and split; custom and percentage splits come in Iteration 3
See balances	BS-001	The balance calculation works on real data
Save and reload data	GE-006	Data survives closing the application

- **User interface:** a simple JavaFX window with a main screen and an Add Expense form, built with Gradle.
- **Navigation:** main screen to the Add Expense form and back. The main screen refreshes the balances after each save.
- **Data storage:** CSV files in a local folder, one file each for groups, members, expenses, and payments, read through a single storage interface.

5.2. Development Approach

- **Start Small:** The shared base (the data model and CSV storage) was built first, so that each feature could be built on top of it without anyone waiting.
- **Iterate Quickly:** Each story is built on its own branch and merged into `dev` through a reviewed pull request as soon as it works, rather than all at once at the deadline.
- **Modularity:** The code is split into four layers (user interface, services, model, and storage), shown in Section 6.1. All calculations live in the services layer, so they can be tested without the screens.
- **Document Your Progress:** Each story is a GitHub issue on our Projects board, grouped into one milestone per iteration. Branches follow the pattern `feat/<member>/<feature>-###-short-description` (for example, `feat/ab/UI-001-balance-dashboard`), and commit messages follow the Conventional Commits format

.

5.3. Task Completion and Team Collaboration

Each story is broken into developer tasks with time estimates. The tasks are to be tracked as a checklist on the story's GitHub issue, and work is tracked on our GitHub Projects board.

Story	Task	Estimate	Owner
GE-006 Keep data after closing	Storage interface and CSV writing	2.5 h	Edward
	Loading CSV data on startup	2 h	
	Tests: save, reload, compare	1.5 h	
GE-001 Create a group and add members	Group and Member classes, create and add logic	1.5 h	Adam
	Name checks (blank, duplicate) and tests	1.5 h	
	Screen to create a group and add members	1 h	
GE-002 Add an expense	Expense class and add logic	2 h	Claire
	Input checks and tests	2 h	
	Add Expense form	2 h	
GE-003 Choose how an expense is split	Split interface and equal split with rounding	2.5 h	
	Custom amount split with total check	2 h	
	Percentage split with 100% check and rounding	2.5 h	

Story	Task	Estimate	Owner
	Tests for all three, including rounding cases	2 h	
	Split method choice on the form	1 h	
GE-004 Edit or delete an expense	Update and delete logic	2 h	
	Tests that balances recalculate	2 h	
	Edit and delete controls in the history	2 h	
GE-005 Remove a member	Removal rule (balance must be zero)	1.5 h	
	Tests	1 h	
	Remove control and blocked message	1.5 h	
BS-001 See my balance	Balance calculation	2.5 h	Edward
	Tests, including uneven splits and payments	2 h	
	Balance display on main screen	1.5 h	
BS-002 See who owes whom	Work out debts from balances	4 h	
	Tests, including zero balances and rounding	3 h	
	Who owes whom list on main screen	3 h	

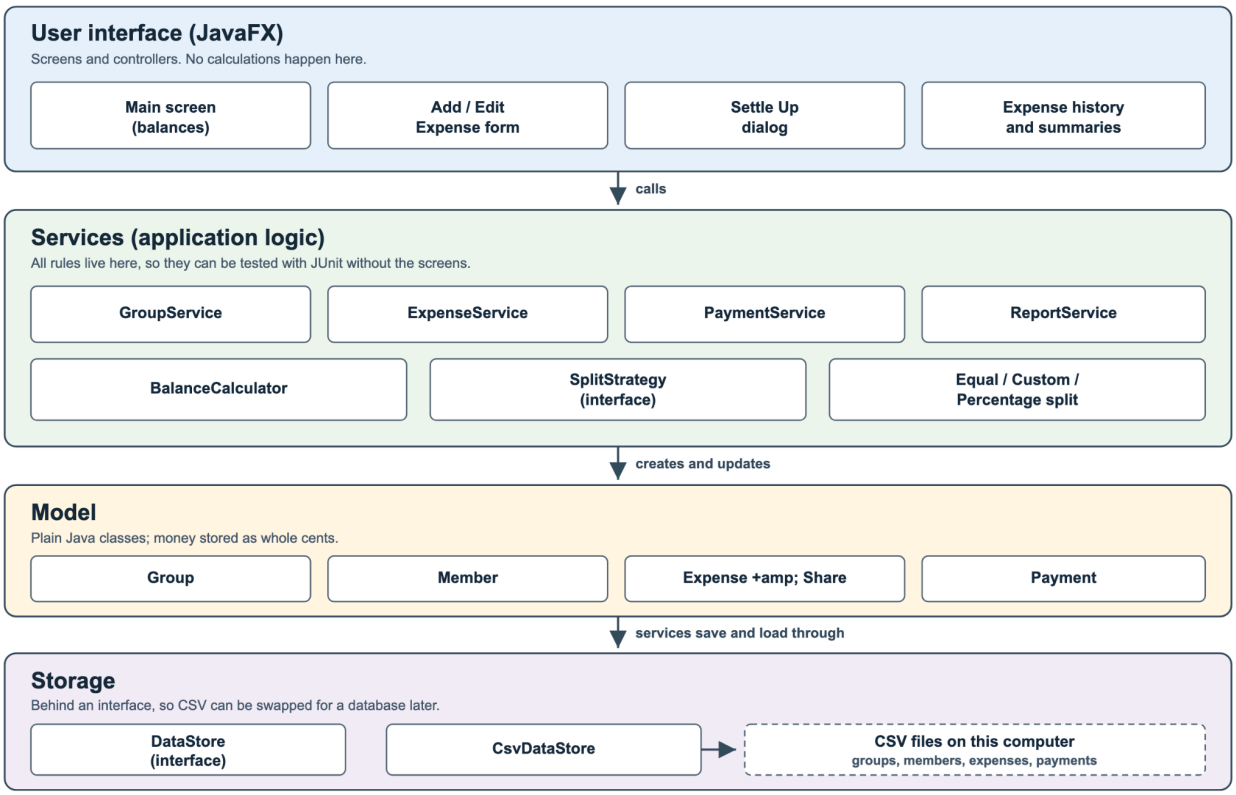
Story	Task	Estimate	Owner
BS-003 Record a payment	Payment class and limit check	2.5 h	
	Tests	1.5 h	
	Settle Up dialog	2 h	
BS-004 Correct a payment	Update and delete logic with recalculation	1.5 h	
	Tests	1 h	
	Edit and delete controls	1.5 h	
UI-002 Clear error messages	Field level error messages	2 h	
	Confirmation messages	1 h	
	Keyboard navigation check	1 h	
UI-001 Balances on opening	Main screen layout	3 h	Francis
	Load and show balances at startup	2 h	
	Empty state message	1 h	
RS-001 Filter past expenses	Filter logic	2.5 h	
	Tests	1.5 h	
	Filter controls	2 h	
RS-002 Spending totals by category	Totals by category and member for a period	2.5 h	
	Tests	1.5 h	

Story	Task	Estimate	Owner
	Summary screen	2 h	
RS-003 Save a balance summary	Build the summary text	1.5 h	
	Save to a chosen file	1 h	
	Tests	1.5 h	
Total	15 stories, 44 story points	**88 h**	

---

6. Documentation Updates

6.1. System Architecture Documentation



SplitEven is organized in four layers, and each layer only talks to the one below it.

- **User interface (JavaFX):** the screens and the code that responds to clicks. It collects input and displays results but does no calculating.
- **Services:** all the rules of the application. The services check input, split expenses, calculate balances, work out who owes whom, and enfoercentage) is its own class behind a shared interface, so a new split method can be added without changing the others.
- **Model:** plain Java classes for groups, members, expenses and their shares, and payments. Money is stored as whole cents to avoid ounding errors.
- **Storage:** reads and writes the model to CSV files on the user's computer. The rest of the application only uses the storage interface, so the CSV files could later be replaced with a database.

Example of all the parts working together: a member saves a new expense on the UI, which gets passed to the expense service. The service uses the chosen split method to calculate each share, and saves the expense into storage. The balance calculator then recalculates balances, and the main screen displays the new totals.

6.2. Planning Document and Log File

Old plan	**Modified plan**
Iteration 1 (Weeks 1 to 2, September 9 to 20): Initial Vision and Planning **Deliverables** ● Project vision document (this document) ● Initial feature categories and high level functionality overview ● Basic repository setup, including the branch structure and a README Iteration 2 (Weeks 3 to 5, September 21 to October 11): Requirements and User Stories **Deliverables** ● Detailed user stories with acceptance criteria, priorities, and story points ● Functional and non functional requirements document ● Use case diagrams and descriptions ● Updated feature backlog, in priority order ● Initial prototype covering the basic structure and one main feature ● Updated plan and change log Iteration 3 (Weeks 6 to 9, October 19 to November 8): Initial Development and Testing **Deliverables** ● Unit tests for the splitting and balance code	**Iteration 1** ● **Week 1-2 (Sep 9 to 20)** ○ Project vision document ○ Initial feature categories and high level functionality overview ○ Basic repository setup, including the branch structure and a README **Iteration 2** ● **Week 3-4 (Sep 21 to Oct 4)** ○ Detailed user stories with acceptance criteria, priorities, and story points ○ Functional and non functional requirements document ○ Use case diagrams and descriptions ○ Updated feature backlog, in priority order ○ Updated plan and change log ● **Week 5 (Oct 5 to 11):** ○ Finalize functional and non functional requirements, user stories, and use cases. ○ Build the shared structure: the data model and CSV storage. **Iteration 3** ● **Week 6 (Oct 12 to 18):** ○ Complete the initial prototype: create a group, add an expense with an equal split, see balances, and save and reload data. ● **Week 7 (Oct 19 to 25):** ○ Review and update the use cases, user stories, backlog, and prototype based on feedback. ● **Week 8 (Oct 26 to Nov 1):**

● Initial prototype covering the high priority user stories ● Testing plan and report, including a table linking each requirement to its test cases	○ Draft deliverable 2.1 to ensure test cases are prepared before writing unit tests. ○ Write unit tests for prototype feature set. ● **Week 9 (Nov 2 to 8):** ○ Feature freeze Thursday November 5, document freeze Friday November 6, submit and tag `ITR2.1` by Sunday November 8.

---

7. Iteration Plan and Backlog

7.1. Iteration Plan

Iteration 1: Initial Vision and Planning

- **Week 1-2 (Sep 9 to 20)**
  - Project vision document
  - Initial feature categories and high level functionality overview
  - Basic repository setup, including the branch structure and a README

Iteration 2: Requirements and User Stories

- **Week 3-4 (Sep 21 to Oct 4)**
  - Detailed user stories with acceptance criteria, priorities, and story points
  - Functional and non functional requirements document
  - Use case diagrams and descriptions
  - Updated feature backlog, in priority order
  - Updated plan and change log
- **Week 5 (Oct 5 to 11):**
  - Finalize functional and non functional requirements, user stories, and use cases. Submit and tag `ITR1.2` by Sunday October 11.
  - Build the shared structure: the data model and CSV storage.

Iteration 3: Initial Development and Testing

- **Week 6 (Oct 12 to 18):**
  - Complete the initial prototype: create a group, add an expense with an equal split, see balances, and save and reload data.
- **Week 7 (Oct 19 to 25):**
  - Review and update the use cases, user stories, backlog, and prototype based on feedback.
- **Week 8 (Oct 26 to Nov 1):**
  - Draft deliverable 2.1 to ensure test cases are prepared before writing unit tests.
  - Write unit tests for the prototype feature set.
- **Week 9 (Nov 2 to 8):**
  - Feature freeze Thursday November 5, document freeze Friday November 6, submit and tag `ITR2.1` by Sunday November 8.

7.2. Updated Backlog

Stories are listed in the order we plan to build them.

Order	Story	Feature	Priority	Points	Target
1	GE-006 Keep data after closing	GE	High	5	Iteration 3 (prototype)
2	GE-001 Create a group and add members	GE	High	3	Iteration 3 (prototype)
3	GE-002 Add an expense	GE	High	5	Iteration 3 (prototype)
4	BS-001 See my balance	BS	High	3	Iteration 3 (prototype)
5	GE-003 Choose how an expense is split	GE	High	3	Iteration 4 (equal split done in Iteration 2)
6	BS-002 See who owes whom	BS	High	3	Iteration 4

Order	Story	Feature	Priority	Points	Target
7	BS-003 Record a payment	BS	High	2	Iteration 4
8	UI-002 Clear error messages	UI	High	5	Iteration 4
9	UI-001 Balances on opening	UI	Medium	8	Iteration 5 (Mid-project, Nov 29)
10	GE-004 Edit or delete an expense	GE	Medium	5	Iteration 5
11	BS-004 Correct a payment	BS	Medium	2	Iteration 5
12	GE-005 Remove a member	GE	Medium	3	Iteration 5
13	RS-001 Filter past expenses	RS	Medium	5	Iteration 5
14	RS-002 Spending totals by category	RS	Low	5	Winter term
15	RS-003 Save a balance summary	RS	Low	3	Winter term